

Documentazione di Progetto: Homework 2

Gestione delle Sedute di Laurea (GUI e Controller)

Gruppo 31

May 26, 2026

1 Introduzione all'Architettura

In questo secondo Homework siamo passati a un'architettura basata sul pattern **MVC (Model-View-Controller)**. L'idea di base era quella di separare nettamente la "mente" dell'applicazione (il package Model, che avevamo già strutturato nel primo homework) dall'interfaccia grafica che vede l'utente finale (il package GUI). Per farli comunicare in modo pulito e senza creare un groviglio di codice, abbiamo introdotto un livello intermedio: il package Controller.

2 Scelte Progettuali del Diagramma UML

Quando abbiamo dovuto riportare questa nuova organizzazione nel nostro **Class Diagram**, abbiamo preso alcune decisioni per rendere il disegno chiaro e il più fedele possibile a quello che succede davvero nel codice:

- **Stereotipi:** Abbiamo aggiunto le etichette «**View**» per le schermate grafiche e «**Controller**» per la classe che fa da intermediario. Ci sembrava il modo migliore per far capire subito, a colpo d'occhio, a quale "mondo" appartiene ogni classe, rispettando le regole standard dell'MVC.
- **Associazioni direzionate (linee continue):** Tutte le nostre View puntano verso il Controller, in quanto ogni finestra ha bisogno di riceverne un'istanza appena viene creata per poter funzionare. Il Controller, ha una freccia fissa verso le classi `Studente`, `Docente`, `Coordinatore` (con cardinalità `0..1`). Questo indica che il Controller si "ricorda" chi sta usando l'app in quel momento (ovviamente all'avvio è vuoto, ecco spiegato lo zero).
- **Dipendenze (linee tratteggiate):** Abbiamo usato le frecce tratteggiate d'uso dal Controller verso entità come `Tirocinio`, `Tesi`, `Richiesta_Tirocinio` e `Seduta_di_Laurea`. In pratica, questo significa che il Controller usa questi oggetti, li crea o li modifica al volo dentro i suoi metodi quando serve, ma non se li tiene salvati per sempre come variabili fisse.

3 Package controller

Il package controller è un po' il "direttore d'orchestra" della nostra applicazione: fa da unico tramite tra le finestre e i dati veri e propri. Al suo interno abbiamo usato un'unica classe principale:

- **Controller:** Si occupa di ricordare chi è collegato in quel momento (salvando l'`utenteLoggato`) e raccoglie i click o i testi inseriti nelle finestre. In base a quello che fa l'utente, il Controller "chiama" i metodi giusti del Model. Abbiamo fatto in modo che le schermate della GUI

non vadano mai a modificare da sole i dati: per qualsiasi cosa, devono sempre chiedere prima al Controller.

4 Package gui

Qui c'è tutta l'interfaccia utente, che abbiamo disegnato usando Java Swing. Ogni volta che apriamo una finestra, le passiamo subito il nostro **Controller**, così sa a chi rivolgersi per fare le cose. Le classi principali che abbiamo sviluppato sono:

- **Main:** È il nostro punto di partenza. Crea il Controller e fa partire la primissima schermata di login.
- **LoginFrame:** È la schermata in cui si inseriscono le credenziali. Una volta riconosciuto l'utente, capisce se si tratta di uno Studente, di un Docente o del Coordinatore e apre la dashboard corretta.
- **StudenteFrame:** L'area dedicata ai ragazzi. Da qui uno studente può chiedere di fare un tirocinio, caricare il file della tesi e, alla fine, prenotarsi per una seduta di laurea.
- **DocenteFrame:** La schermata per i professori. Permette loro di proporre nuovi argomenti per i tirocini e di fare da "giudici": decidono se accettare o scartare le richieste degli studenti e valutano le tesi finali.
- **CoordinatoreFrame:** Il pannello di controllo amministrativo. Serve a creare fisicamente gli appelli di laurea sul calendario e ad assegnarci i professori che formeranno la commissione.

5 Package model (Dizionario Entità aggiornato)

Per quanto riguarda il Model, siamo rimasti molto fedeli a come lo avevamo pensato all'inizio. Le fondamenta del progetto si basano su queste entità:

Nome Entità	Descrizione
Utente	La classe base da cui partiamo. Contiene i dati anagrafici e le credenziali di accesso.
Studente	"Figlio" di Utente. È chi interagisce di più con tirocini, tesi e appelli.
Docente	"Figlio" di Utente. Propone le idee per i tirocini, fa da relatore e valuta i lavori.
Coordinatore	Un Docente con poteri organizzativi extra, come la creazione delle sedute di laurea.
Tirocinio	Il classico tirocinio universitario interno proposto da un prof.
TirocinioEsterno	Un tirocinio fatto in azienda (salviamo anche il nome dell'Azienda e del Referente).
Richiesta_Tirocinio	Collega uno Studente, un Docente e un Tirocinio. Ha uno stato per capire se è stata approvata o meno.
Tesi	Il documento caricato dallo studente in attesa di essere letto e valutato.
Seduta_di_Laurea	L'evento vero e proprio, con data, ora, aula e professori assegnati in commissione.
Prenotazione_Laurea	È la "ricevuta" di iscrizione dello studente a un determinato appello.

5.1 Dizionario degli Stati (Enumerazioni)

Per evitare di fare confusione con stringhe scritte a mano e gestire meglio i vari "semafori" logici del programma, ci siamo appoggiati a queste tre enumerazioni:

- **StatoRichiesta:** RIFIUTATO, ACCETTATO, IN_ATTESA.
- **StatoTesi:** ACCETTATO, RIFIUTATO, IN_ATTESA.
- **StatoLaurea:** ACCETTATO, RIFIUTATO, IN_ATTESA.